Ring Wars

Game Design Document | Final Draft

Jim Powell | Multi Agent AI for Game Development
Instructor: Joshua Burdick | Last updated: 2026-10-19

Full GDD (Markdown on GitHub)

Executive Summary

Ring Wars is a singleplayer, turn-based tactical squad shooter set in the Iris Expanse. **The player is the squad**, directing a standard team of **4 operators** fighting the Directorate's occupation of Marendel. The Nova resistance raids communications, power, supply routes, prisons, and command facilities, gradually turning a foothold into a campaign to liberate the planet.

Element	Design
Platforms and controls	Windows and macOS; mouse and keyboard
View	Elevated 3D battlefield, free tactical camera, and directed action shots
Audience	Players who enjoy coordinated turns, squad building, and developing familiar characters
Campaign	Authored story operations, mission choices, and generated side missions

Player experience

The squad is capable and outnumbered. The player succeeds by finding an angle, combining specialists, and preserving a way home. Shots within a weapon's optimal range hit with certainty; accuracy falls beyond it toward maximum range. Aiming shows hit chance and damage. Advancing can secure reliable fire, while cover and armor reduce the damage of hits.

Each Act introduces an ally: the handler who frees the opening prisoner, a gruff man who dislikes the squad, a pragmatic man who likes it, and Nova's CEO and resistance leader. Act IV reunites their squads for the capital assault. The finale costs the leader's life and includes a player-directed sacrifice with consequences for a possible DLC.

Between operations, headquarters connects the War Table, Barracks, and Armoury. The player chooses missions, equips the squad, trains abilities, and develops recognizable operators through callsigns, appearance, and specialization. Captured weapons and rescued specialists expand those choices through the campaign's events.

Victory and defeat

The campaign culminates in defeating occupation command in the capital. A mission succeeds when its objective is completed and the squad extracts, or when the defending force is defeated and the location secured, as required by that operation.

A main-story wipe is **Game Over**; retrying returns to before the failed attempt. A side-mission wipe closes that operation: each operator has a **45% capture chance**, otherwise they are permanently lost. Resolved missions leave the campaign's available set. During successful extraction, operators left outside the zone are lost.

Developed by Jim Powell with AI assistance.

Game Mechanics

Core loop

1. **Choose an operation.** Compare objectives, conditions, and rewards at the War Table.
2. **Prepare the squad.** Select operators, equip kits and gear, and train abilities.
3. **Execute coordinated turns.** Use routes, positioning, and the squad's actions to complete the objective.
4. **Resolve the operation.** Extract under reinforcement pressure or secure the site.
5. **Develop the resistance.** Return to headquarters, use rewards, spend experience, and prepare again.

Actions and tactical consequences

Combat alternates between **Our Turn** and **Their Turn**. During Our Turn, the player can switch freely among operators and spend action points (AP) on movement, attacks, abilities, revival, and objective interactions. Moving, attacking, and moving again are valid choices. End Turn commits the squad to the enemy response.

Movement previews show routes and AP costs. Directional cover absorbs incoming damage from the protected angle, while a flank can bypass it. Armor protects the unit itself. Destructible cover can become exposed during a firefight, creating a new opportunity or making a previously useful position dangerous.

Burst weapons check each round separately. Overwatch is limited to optimal range and delivers guaranteed hits. Suppressing Fire can stop movement through its cone. The player counters a defended lane by changing angle, breaking sightlines, using smoke, or attacking the watcher. Enemy grenades mark a delayed blast area, giving the player a turn to reposition.

Operators and recovery

Lost or captured operators are immediately replaced by recruits with a pistol and starting XP. Credits outfit them; losses cost veterancy, identity, and equipment investment. A rescued veteran replaces a current operator, who is permanently reassigned to another unit. Roster capacity stays filled. Achievements and ranks record individual growth.

Every operator can revive an in-range Downed teammate. Medkits unlock in the first squad mission; the Medic makes healing and revival easier to sustain.

Example of play

In **Supply Sabotage**, the squad plants charges on a Directorate stockpile and extracts. A sentry covers the direct approach. The Sword operator takes a longer flank while the Rifle operator covers movement. The Sniper attacks an exposed defender; the result informs whether another operator finishes that target or advances.

Once charges are planted, reinforcements make withdrawal urgent. An optional cache adds reward, distance, and exposure. The player weighs that detour, conceals a crossing with smoke, and creates room to revive a Downed teammate. Success rewards the attack plan and the route preserved for getting home.

Authored operations carry the resistance story. Generated side missions combine authored spatial blocks, objectives, and enemy placements into varied tactical sites, with each offer's setup fixed before deployment.

AI Architecture

Specialized development agents turn prose briefs into content and checked changes. Their work becomes the places, performances, encounters, and interactions the player experiences. Unity executes the tactical rules, enemy behavior, and generated mission assembly.

Voiceover Agent

When approved dialogue changes, this agent generates and integrates spoken lines so the player hears the mission briefing or exchange with matching subtitles.

Input: Approved dialogue, speaker identity, and delivery direction. **Structured output:** Line identifiers, clip references, subtitle associations, durations, and playback findings.

Level Designer Agent

When a mission brief is ready, this agent designs routes, objectives, cover, and visual references so the player has useful choices about approaching the target and withdrawing.

Input: Mission purpose, setting, enemy roles, insertion, and extraction needs. **Structured output:** Layout and brief references, route and objective annotations, fixed-camera reference views, and layout checks.

Level Builder Agent

When a level design is approved, this agent sources assets and builds the playable location so the player can move, fight, use cover, and complete objectives inside a coherent Marendel facility.

Input: Approved layout, visual references, and asset sources. **Structured output:** Source asset references, Blender and Unity deliverables, and import, navigation, cover, lighting, and playable-path findings.

Balance Agent

When a kit, encounter, reward, or playtest finding needs adjustment, this agent revises the relevant definitions so the player has useful squad choices and encounters that reward coordinated action.

Input: A specific gameplay issue and its intended experience. **Structured output:** Changed definition identifiers, the change set, and before-and-after gameplay observations.

Testing Agent

When a playable change is ready, this agent checks the affected path and reports reproducible failures so the player can deploy, complete the mission, extract, and use the rewards.

Input: The changed feature and expected player-visible behavior. **Structured output:** Expected and observed results, pass or fail status, reproduction steps, and evidence.

One Wow agent

The **Level Builder Agent** produces the most visible result: a playable Marendel location. In Supply Sabotage, the player must recognize the stockpile, read a protected approach, find useful firing positions, and see a route to extraction. Lighting, materials, destructible cover, and surrounding scenery make those decisions part of a believable occupied facility.

Technical Strategy

Foundation and production scope

Ring Wars uses **Unity 6 URP**, C# gameplay systems, editable content definitions, and Blender. The level-building system is **complete, tested, and approved**. Agents regularly run tests and mission-to-headquarters playthroughs using the existing Editor tools.

The eight-week deliverable is a **full singleplayer campaign playthrough** with alpha cinematics, alpha voiceover, and slim cosmetic options. Warmech, Medic, and Captain are cuts if needed; affected missions remain playable with available equipment. Polish and additional settings follow as time allows.

Agent contracts and handoffs

Each agent accepts a prose brief and returns a **JSON result manifest** identifying the task, agent, status, summary, inputs, deliverables, checks, and open questions. Native files remain the working artifacts. The Level Designer's package feeds the Level Builder; Voiceover uses approved dialogue; Balance and Testing evaluate the playable result. Jim directs priorities and reviews visual quality, timing, and feel.

Biggest technical constraint

Art production is the main constraint. Environments and operators must share a coherent look while supporting cover, destruction, navigation, and close camera views. Licensed packs, shared Blender libraries, modular pieces, and the asset generation pipeline supply reusable foundations. Checked layouts and fixed-camera references guide construction; generated concepts, selected textures, and voice performances adapt the assets to Marendel. Jim reviews the result through gameplay cameras.

Token budget

The initial estimate is **1,000,000 text-agent tokens per week**, or **8,000,000 over eight weeks**. A reference production cycle budgets:

Agent task	Tokens
Voiceover preparation and integration	15,000
Mission layout and reference pass	80,000
Level build and Unity integration	160,000
Focused balance pass	35,000
Mission and campaign-path verification	30,000
Total per cycle	**320,000**

Two reference cycles use 640,000 tokens, leaving 360,000 weekly for integration and rework. A cycle can revise an existing mission. These usage estimates include input, output, and reported reasoning. Image and speech costs are tracked separately. Gameplay uses **zero generative-AI tokens per mission**.

API constraints

Text agents use the connected account's request, token, and context limits, with focused briefs and bounded concurrency. JSON handoffs are checked. ElevenLabs generation respects credits and concurrency, reusing unchanged lines. Image generation uses focused batches and retained references. The game uses imported media and authored assets.

API references: OpenAI rate limits, structured outputs, and ElevenLabs API errors.